

PHISHGUARD

Project Documentation

Intelligent Email Phishing Analyzer

Document Purpose

This summary presents the project in a concise academic format. It focuses on the problem, design decisions, main features, implementation approach, testing, and deployment readiness without repeating every technical detail from the full report.

Item	Summary
Project Area	Cybersecurity, phishing detection, web application development
System Type	Full-stack web platform with rule-based analysis and NLP-assisted machine learning
Main Users	General users, security analysts, and administrators
Submission Material	Source code, documentation, presentation, tests, and reports

Contents

- 1. Executive Summary
- 2. Project Overview
- 3. Requirements Summary
- 4. System Design
- 5. Implementation Summary
- 6. Testing and Quality Assurance
- 7. Deployment and User Guide
- 8. Conclusion

The contents are intentionally short so the document can be reviewed quickly while still covering the required academic areas.

1. Executive Summary

PhishGuard is a full-stack cybersecurity project designed to help users examine suspicious emails, links, headers, and uploaded files before trusting them. The system gives a clear risk score, explains the evidence behind the result, and stores each analysis so it can be reviewed, exported, or corrected later.

The project combines practical security rules with an NLP-based machine learning layer. This keeps the system useful even when a trained model is not available, while still allowing the model to improve through approved feedback. The final product is a working web application with authentication, role-based access, reporting, administration, traceable feedback, and repeatable execution support.

Key Outcome

The project delivers a usable phishing-analysis platform that presents technical evidence in a way normal users and reviewers can understand.

2. Project Overview

Phishing remains difficult to detect because attacks often mix technical tricks with human persuasion. A message may look familiar, use urgent language, hide a dangerous URL, or imitate a trusted sender. PhishGuard addresses this problem by checking several signals together instead of depending on one indicator.

2.1 Objectives

- Provide a safe way to analyze suspicious emails, URLs, headers, and files.
- Explain why a message is safe, suspicious, phishing, or critical.
- Support normal users, analysts, and administrators with clear permissions.
- Store analysis history and generate PDF, JSON, and CSV reports.
- Use feedback to support future dataset and model improvement.

2.2 Project Scope

The scope covers the complete application workflow: registration, login, analysis, result explanation, history management, reports, feedback review, user administration, model governance, audit logs, and system health checks. The system also includes local and Docker-based execution guidance so it can be reviewed consistently.

3. Requirements Summary

The requirements were shaped around the actual tasks a reviewer or user would perform. The application needs to be fast enough for repeated checks, clear enough for non-specialists, and strict enough to avoid unsafe behavior when handling suspicious content.

Area	Requirement Summary
Authentication	Users can register and log in; protected pages require valid access tokens.

Analysis	The system accepts pasted email content, uploaded files, raw headers, and URLs.
Result Display	Each result includes a risk score, classification, explanation, and recommended action.
History	Users can search, filter, re-analyze, delete, and export prior analyses.
Reports	Reports show historical trends, phishing rate, high-risk items, and export options.
Admin	Administrators manage users, feedback, model status, audit logs, and health checks.
Security	Uploads are limited, dangerous URL behavior is avoided, and sensitive actions are protected.

3.1 Roles

Role	Main Responsibility
User	Run analyses and review personal history.
Analyst	Review findings, feedback, and reports.
Admin	Manage accounts, model workflow, settings, audit records, and system health.

4. System Design

PhishGuard follows a layered design. The frontend is responsible for usability and presentation, while the backend handles authentication, validation, analysis, persistence, reporting, and administrative workflows. The database keeps users, analyses, indicators, URLs, attachments, feedback, model versions, and audit logs separated for easier maintenance.

4.1 Architecture

Layer	Description
Frontend	Next.js interface with dashboard, analyzer, history, reports, feedback, settings, and admin pages.
Backend	FastAPI services for authentication, analysis, reports, feedback, administration, and health checks.
Analysis Engine	Rule-based analyzers for URLs, sender identity, headers, attachments, language, and authentication evidence.
Machine Learning	TF-IDF and structured cybersecurity features used by a logistic regression model.
Database	SQLAlchemy models with PostgreSQL-ready persistence and migration support.
Reporting	PDF, JSON, and CSV outputs for analysis and historical reports.

4.2 Main Analysis Flow

1. The user submits suspicious content through the Analyzer page.
2. The backend validates the input and safely extracts evidence.

3. The rule engine scores URL, sender, header, attachment, and language signals.
4. The NLP model contributes a probability-based confidence score when available.
5. The system stores the result and displays a readable explanation to the user.
6. Feedback can be reviewed by an administrator before it joins the verified dataset.

5. Implementation Summary

The implementation is organized as a real software project rather than a single isolated page. The frontend and backend are separated, configuration is handled through environment variables, and the project includes tests, migrations, Docker support, and documentation for running the system locally.

Component	Technology
Frontend	Next.js, TypeScript, Tailwind CSS, Recharts, Lucide icons
Backend	FastAPI, Pydantic, SQLAlchemy, Alembic
Database	PostgreSQL-ready schema with local development support
Authentication	JWT access tokens and Argon2 password hashing
Machine Learning	scikit-learn, TF-IDF, engineered features, joblib artifacts
Reports	PDF/JSON/CSV export services
Deployment	Docker Compose with frontend, backend, and database services

5.1 Notable Features

- Clean analyzer workflow with tabs for email, file, URL, and raw headers.
- Risk score scale from Safe to Critical with clear recommended action.
- Weekly report charts that avoid misleading percentage spikes on small samples.
- History filters, export options, and re-analysis support.
- Admin tabs for users, feedback queue, model and dataset, audit logs, and system health.
- Toast notifications, loading states, empty states, and clear error messages.

5.2 Latest Implementation Status

The latest implementation pass focused on making review evidence accurate, keeping admin workflows useful, and aligning the deliverables with the current repository.

- Feedback submitted by regular users now appears in the Admin Feedback Queue with submitter name, email, pending status, manual refresh, and periodic refresh.
- History and CSV exports preserve the original analysis owner, so administrator review no longer relabels user analyses as admin-owned records.
- Correct-result feedback is normalized before dataset approval, which keeps future training samples consistent.
- Cloud-specific settings were removed from the application package; local, Docker, and GitHub-based review remain the documented execution path.

6. Testing and Quality Assurance

Testing focuses on the flows that prove the platform works as a product: account access, analysis submission, result display, exports, feedback, administration, charts, responsive behavior, and owner-safe review. Automated checks are supported by manual review of the main pages.

Check	Result
Backend tests	20 tests passed
Frontend unit tests	7 tests passed
Linting	Passed
Production build	Passed
Browser tests	5 passed, 1 intentionally skipped in the suite
Security audit	No high-level npm vulnerabilities reported
Latest workflow fixes	Feedback queue and owner-safe history verified

6.1 Quality Notes

- The system avoids treating missing data as a successful model result.
- Low-volume report periods are handled carefully so percentages are not misleading.
- Administrative actions are protected by role checks and confirmation patterns.
- File upload handling uses limits and known file-type controls.

7. Deployment and User Guide

The project can be run locally for development or through Docker Compose for a cleaner review environment. The first administrator can be seeded from environment variables, which makes setup repeatable without hardcoding users into the application.

7.1 Basic User Workflow

7. Register or log in to the application.
8. Open Analyzer and choose the input method.
9. Submit suspicious content for analysis.
10. Review the score, explanation, risk contributions, and recommended action.
11. Export the report or submit feedback if the result needs review.

7.2 Administrator Workflow

12. Log in with an administrator account.
13. Review users, feedback queue, audit logs, model status, and system health.
14. Approve valid feedback so it can support the verified dataset.
15. Train or activate model candidates only when enough approved samples exist.

Deployment Readiness

The repository is small enough for GitHub after excluding generated folders such as `node_modules` and `.next`. These folders are recreated with `npm install` and `npm run build`.

8. Conclusion

PhishGuard meets the main academic and practical goals of the project. It gives users a safe place to check suspicious messages, presents evidence in a readable way, and gives administrators the tools needed to manage accounts, feedback, model readiness, reports, and system health.

The most important strength of the project is that it connects security analysis with a complete user workflow. The application does not stop at producing a score; it explains the result, stores the evidence, supports review, and prepares the project for future improvement through a verified feedback dataset.